چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۱۸

کارایی — بخش ۲: حجم اپ، باتری، Baseline Profile و Macrobenchmark



اسماعیل مرادی

@esmaeeil-moradi700

چرا این بخش در ۲۰۲۶ مهم است؟

بخش ۱ دربارهٔ فریم و حافظه بود. بخش ۲ دربارهٔ همه چیزِ اطراف آن‌هاست: بایت‌هایی که کاربر دانلود می‌کند، باتری و اینترنتی که مصرف می‌کنید، سرعت کد شما از روز اول — و اینکه گوگل در میدان چطور قضاوتتان می‌کند.

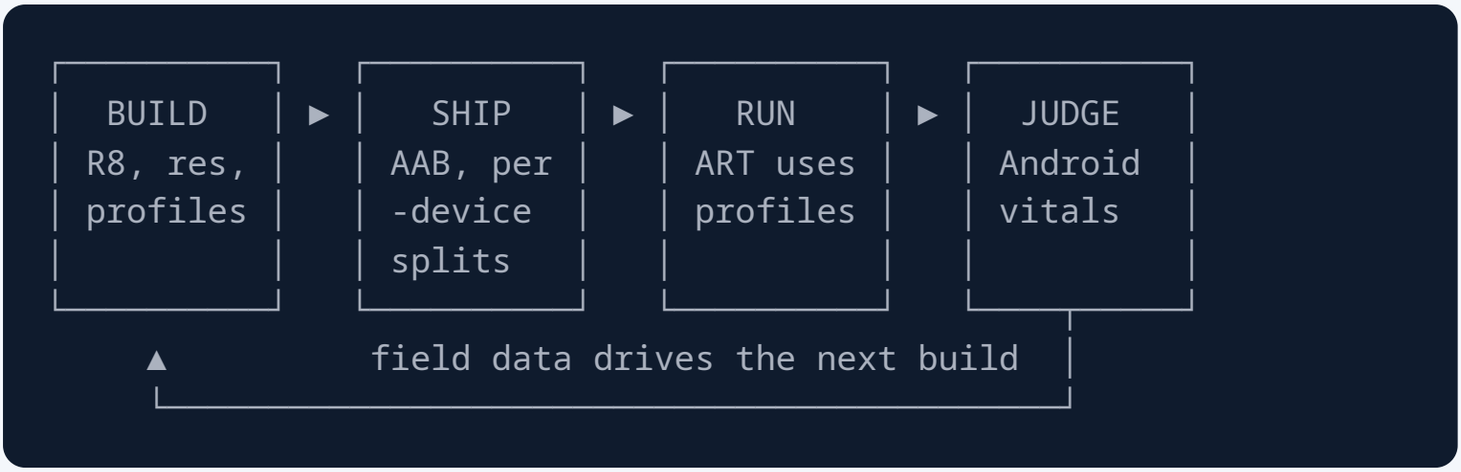
- حالت full در R8 از AGP 8.0 پیش‌فرض است — و چیزی را که keep ruleها باید بگویند عوض می‌کند
- Google Play از App Bundle شما split APK می‌سازد؛ فایلی را که آپلود می‌کنید هیچ‌کس دانلود نمی‌کند
- باتری یک سیگنال کیفیت است: Android vitals در کنار crash و ANR، wake lockهای بیش از حد را هم گزارش می‌کند
- Baseline Profile و Startup Profile کندی اجرای اول را درست می‌کنند؛ کاری که از JIT برنمی‌آید
- Macrobenchmark تغییر را در آزمایشگاه ثابت می‌کند؛ Android vitals آن را در production تأیید می‌کند

سطح	سؤال‌ها
جونیور	چه چیزی اپ را بزرگ می‌کند، App Bundle، چه چیزی باتری را می‌خورد
مید	حالت full در R8، اهرم‌های حجم، بهره‌وری شبکه، ART کد شما را چطور اجرا می‌کند
سینیور	Baseline و Startup Profile، Macrobenchmark به‌طور عمیق، Android vitals
آرکیتکت	حجم به‌عنوان معماری، همگام‌سازی کم‌مصرف، ابزارهای آزمایشگاه و میدان



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: build ← انتشار ← اجرا ← قضاوت



- **build:** کامپایلر R8 کد و resourceها را کوچک می‌کند؛ Baseline Profile و Startup Profile داخل اپ بسته‌بندی می‌شوند
- **انتشار و اجرا:** Play فقط چیزی را که هر دستگاه لازم دارد می‌فرستد؛ ART مسیرهای داغ را از روی profile کامپایل می‌کند
- **قضاوت:** vitals کاربران واقعی نسخهٔ بعد را جهت می‌دهد؛ Macrobenchmark در آزمایشگاه از هر مرحله محافظت می‌کند

بده‌بستان را نام ببرید

حجم، باتری و سرعت با هم کشمکش دارند: کامپایل کامل AOT سریع است اما حجیم؛ prefetch سریع است اما اینترنت مصرف می‌کند. جواب سینیور می‌گوید چه چیزی را با چه چیزی معاوضه می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

چه چیزی اپ را بزرگ می‌کند، App Bundle، چه چیزی باتری را می‌خورد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. چه چیزی اپ را بزرگ می‌کند؟

داخل APK	چیست	اهرم اصلی
`classes*.dex`	کد شما به‌علاوهٔ همهٔ کتابخانه‌ها	R8
`res/` و `resources.arsc`	layoutها، تصویرها، رشته‌ها	resource shrinking، vector، WebP
`lib/<abi>/`	کتابخانه‌های native با پسوند `.so`	فقط ABIهایی که دستگاه لازم دارد
`assets/`	فایل‌های خام: مدل ML، فونت، رسانه	Play Asset Delivery، دانلود بعدی

- **APK Analyzer** در منوی Build، با گزینهٔ Analyze APK: حجم خام در برابر حجم دانلود، مقایسهٔ دو build، نگاه به داخل فایل‌های dex
- **حجم دانلود** (فشرده، همان چیزی که کاربر منتظرش می‌ماند) با **حجم روی دستگاه** فرق دارد؛ دومی با کد کامپایل‌شده و داده بزرگ‌تر می‌شود

تلهٔ مصاحبه

اندازه گرفتن حجم یک APK **دیباگ**. buildهای دیباگ نه کوچک می‌شوند نه بهینه — فقط build ریلیز چیزی به شما می‌گوید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. App Bundle: بگذارید Play تقسیم کند

```
app-release.aab ──► Google Play ──► this device installs:
 (upload key)       (signs with      base.apk
                     app signing     split_config.arm64_v8a.apk
                     key)            split_config.xxhdpi.apk
                                     split_config.fa.apk
```

- از اوت ۲۰۲۱ برای اپ‌های جدید در Google Play الزامی است
- **Play App Signing:** شما آپلود را با یک *upload key* امضا می‌کنید؛ Play خودش APKهایی را که می‌سازد امضا می‌کند

```
# Per-device sizes from the bundle you upload
bundletool build-apks --bundle=app-release.aab \
  --output=app.apks
bundletool get-size total --apks=app.apks
```

تلهٔ مصاحبه

«فایل `.aab` من ۴۰ مگابایت است، پس کاربر ۴۰ مگابایت دانلود می‌کند.» — هیچ‌کس bundle را دانلود نمی‌کند. Play برای هر دستگاه split APK می‌سازد — یک ABI، یک density و زبان‌های کاربر — آن هم فشرده. با bundletool یا بخش app bundle explorer در Play Console اندازه بگیرید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. دو تله در bundle

تلهٔ مصاحبه

«*برای کم کردن حجم، در Gradle برای هر ABI و density یک APK جدا بسازید.*» — در Play، bundle همین کار را برای هر دستگاه انجام می‌دهد. بلوک `splits {}` در Gradle فقط وقتی مهم است که خودتان APK پخش می‌کنید (استورهای دیگر، دانلود مستقیم).

تلهٔ مصاحبه

«*تقسیم زبان‌ها هزینه‌ای ندارد.*» — اگر داخل اپ انتخاب زبان دارید، زبان‌هایی که دستگاه استفاده نمی‌کند نصب نمی‌شوند. یا در زمان اجرا با `SplitInstallManager` درخواستشان کنید، یا تقسیم زبان را خاموش کنید:

```
android {
    bundle {
        language { enableSplit = false }
    }
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. چه چیزی باتری را می‌خورد

مصرف‌کننده	چرا	بهتر
partial wake lock	CPU نمی‌تواند بخوابد	WorkManager که خودش lock را نگه می‌دارد، یا یک lock کوتاه و زمان‌دار
شبکهٔ پرتکرار	هر انتقال رادیو را بیدار می‌کند و رادیو تا مدتی بعد در حالت پرمصرف می‌ماند	دسته‌بندی، prefetch، push به‌جای poll
موقعیت مکانی	GPS گران است	کمترین priority که کار را راه می‌اندازد؛ به‌روزرسانی دسته‌ای
بیدارباش‌ها	alarmها و jobهایی که دستگاه را بیدار می‌کنند	alarm غیردقیق، constraintهای WorkManager (روزهای ۱۵ و ۱۶)
CPU در پس‌زمینه	کاری که کسی نمی‌بیند	با lifecycle متوقفش کنید

ابزارها: **Power Profiler** در Android Studio که مصرف ریل‌های برق را روی دستگاه‌های پشتیبانی‌شده (مثلاً Pixel 6 و جدیدتر) اندازه می‌گیرد، دستور `adb shell dumpsys batterystats` و Android vitals.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. اگر wake lock لازم دارید، زمان‌دارش کنید

```
val pm = context.getSystemService(PowerManager::class.java)
val lock = pm.newWakeLock(
    PowerManager.PARTIAL_WAKE_LOCK, "myapp:upload",
)
lock.acquire(10 * 60 * 1000L) // always with a timeout
try {
    uploadPendingPhotos()
} finally {
    if (lock.isHeld) lock.release()
}
```

- بهتر از آن: اصلاً wake lock خودتان را نداشته باشید — WorkManager تا وقتی `doWork()` اجرا می‌شود دستگاه را بیدار نگه می‌دارد
- برچسب را به شکل `app:purpose` بگذارید تا در `batterystats` و vitals قابل‌شناسایی باشد

تلهٔ مصاحبه

«*Doze باگ‌های wake lock را بی‌خطر می‌کند.*» — Doze عمیق فقط وقتی شروع می‌شود که دستگاه مدتی از شارژ جدا، بی‌حرکت و با صفحهٔ خاموش مانده باشد. تا آن موقع lock شما واقعاً CPU را بیدار نگه می‌دارد — و Android vitals هم آن را حساب می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

حالت full در R8، اهرم‌های حجم، بهره‌وری شبکه، ART کد شما را چطور اجرا می‌کند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. R8: کوچک کردن، بهینه‌سازی، مبهم‌سازی

```
android {
    buildTypes {
        release {
            isMinifyEnabled = true    // R8 on code
            isShrinkResources = true  // then unused resources
            proguardFiles(
                getDefaultProguardFile(
                    "proguard-android-optimize.txt"
                ),
                "proguard-rules.pro",
            )
        }
    }
}
```

- کد بی‌استفاده را **حذف می‌کند**، **بهینه می‌کند** (inline کردن، ادغام کلاس‌ها، unbox کردن enum)، نام‌ها را **مبهم می‌کند**؛ بعد resourceهای بی‌استفاده حذف می‌شوند
- **حالت full از AGP 8.0 پیش‌فرض است** — فرض‌های کمتری از حالت سازگار با ProGuard می‌کند

تلهٔ مصاحبه

«*هر فایل پیش‌فرض ProGuard کار را راه می‌اندازد.*» — فایل `proguard-android.txt` شامل `-dontoptimize` است: کوچک‌سازی بدون بزرگ‌ترین بردهای R8. و R8 «فقط مبهم‌سازی» نیست — کوچک‌سازی و بهینه‌سازی اپ را کوچک‌تر **و** سریع‌تر می‌کنند (امنیت: روز ۲۰).



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. حالت full و keep ruleها

حالت full چیزهایی را که reflection *شاید* لازم داشته باشد نگه نمی‌دارد: keep کردن یک کلاس، constructor پیش‌فرضش را نگه نمی‌دارد، و attributeهای `Signature` فقط روی چیزهای keepشده می‌مانند. مدل‌های Gson و کلاس‌هایی که با نام بارگذاری می‌شوند rule صریح لازم دارند.

```
# Narrow and commented. Never: -keep class com.example.** { *; }
-keep class * implements com.example.Plugin {
    <init>();
}
# Why is this still in the APK?
-whyareyoukeeping class com.example.feed.FeedMapper
```

- کتابخانه‌ها consumer rule خودشان را همراه دارند؛ کتابخانه‌هایی را ترجیح دهید که به reflection نیاز ندارند
- جست‌وجوی پویا با `getIdentifier()` باعث می‌شود resource shrinking محتاط شود؛ فایل `res/raw/keep.xml` با `tools:keep` و `tools:discard` آن را کنترل می‌کند

تلهٔ مصاحبه

«حالت full ریلیز را خراب کرد، پس خاموشش کن.» — تنظیم `android.enableR8.fullMode=false` مشکل را پنهان می‌کند. کد reflectionدار را پیدا کنید (از روی stack و `mapping.txt`)، یک rule دقیق اضافه کنید و همان مسیر را روی build ریلیز تست کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. اهرم‌های حجم فراتر از R8

اهرم	چه کنیم
تصویرها	vector drawable برای آیکون‌ها؛ WebP برای عکس‌ها (Studio می‌تواند تبدیل کند)
وابستگی‌ها	تغییر حجم build ریلیز را برای هر SDK جدید بسنجید — یک SDK ممکن است از یک فیچر سنگین‌تر باشد
reflection	کتابخانهٔ `kotlin-reflect` و کتابخانه‌های JSON مبتنی بر reflection سنگین‌اند و سخت کوچک می‌شوند
کد native	فقط ABIهای لازم (bundle جدایشان می‌کند)؛ حذف symbolهای دیباگ
assetهای بزرگ	مدل ML و ویدیو: Play Asset Delivery یا دانلود در صورت نیاز

ANDROID 15+ · 16 KB PAGES

Google Play از اپ‌هایی که کد native دارند و Android 15 به بالا را هدف می‌گیرند، پشتیبانی از **صفحه‌های حافظهٔ ۱۶ کیلوبایتی** را می‌خواهد. فایل‌های `.so` را با NDK و AGP به‌روز دوباره build کنید (نسخه‌های اخیر هر دو به‌طور پیش‌فرض برای ۱۶ کیلوبایت تراز می‌کنند) و از سازندگان SDK نسخهٔ به‌روز بگیرید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. دو افسانه دربارهٔ حجم

تلهٔ مصاحبه

«*از Gson یا Moshi استفاده کن، سبک‌اند.*» — Gson مبتنی بر reflection است: کندتر، و زیر حالت full در R8 شکننده. serializerهای زمان کامپایل را ترجیح دهید: kotlinx.serialization، یا Moshi با codegen.

تلهٔ مصاحبه

«*برای صرفه‌جویی در حجم، enumها را با ثابت‌های* `Int` *عوض کن.*» — توصیه‌ای مال یک دهه پیش. R8 enumهای ساده را unbox می‌کند. کد خوانا بنویسید و build ریلیز را اندازه بگیرید.

سؤال تکمیلی

«*چطور پیدا می‌کنید چه چیزی بزرگ شده؟*» — APK ریلیز قدیم و جدید را در APK Analyzer مقایسه کنید: فایل به فایل dex، resourceها و کتابخانه‌های native را diff می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. بهره‌وری شبکه: بیدارباش کمتر، بایت کمتر

- درخواست‌ها را **دسته‌بندی** کنید و چیزی را که کاربر احتمالاً لازم دارد وقتی رادیو روشن است **از قبل بگیرید**
- **poll نکنید** — با FCM یک سیگنال push کنید و یک بار sync کنید (روز ۱۶)
- **کمتر بفرستید:** صفحه‌بندی، انتخاب فیلدها، تصویر در اندازهٔ نمایش از CDN
- **فشرده‌سازی و cache:** کتابخانهٔ OkHttp خودش gzip را شفاف مدیریت می‌کند؛ HTTP caching موضوع روز ۱۱ است
- **به شبکهٔ کاربر احترام بگذارید:** اتصال‌های metered و Data Saver
- ترافیک واقعی را با **Network Inspector** در Android Studio ببینید

تلهٔ مصاحبه

«*poll کردن هر دقیقه مشکلی ندارد، payload کوچک است.*» — هزینه، بیدار شدن رادیو و دنبالهٔ پرمصرف بعد از آن است، نه بایت‌ها. شصت درخواست کوچک در ساعت یعنی شصت بار بیدار شدن رادیو.

تلهٔ مصاحبه

«*یک interceptor برای gzip به OkHttp اضافه کن.*» — OkHttp خودش gzip را درخواست و از حالت فشرده خارج می‌کند — مگر اینکه خودتان `Accept-Encoding` را تنظیم کنید؛ آن وقت decompress کردن با شماست.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. خودتان را با شبکهٔ کاربر تطبیق دهید

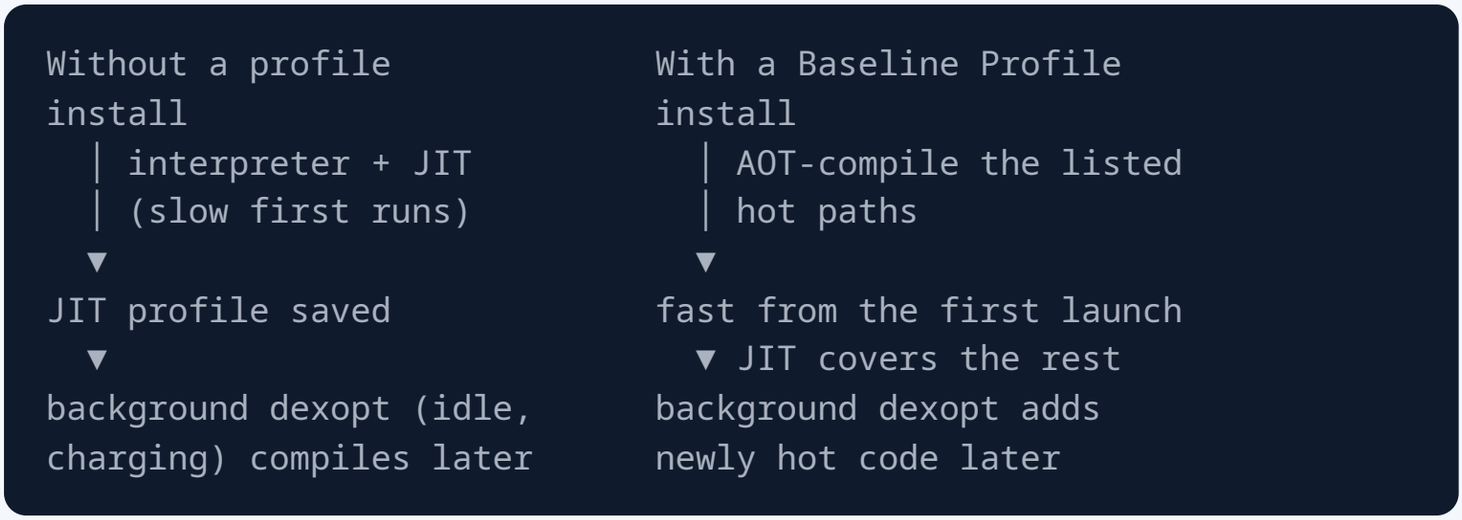
```
fun prefetchPolicy(cm: ConnectivityManager): Prefetch {
    val caps = cm.getNetworkCapabilities(cm.activeNetwork)
        ?: return Prefetch.None
    val dataSaver = cm.restrictBackgroundStatus ==
        RESTRICT_BACKGROUND_STATUS_ENABLED
    return when {
        dataSaver -> Prefetch.None
        caps.hasCapability(NET_CAPABILITY_NOT_METERED) ->
            Prefetch.Full
        else -> Prefetch.ThumbnailsOnly
    }
}
```

- انتقال‌های سنگین پس‌زمینه: WorkManager با `NetworkType.UNMETERED` ، و شاید `setRequiresCharging(true)`
- به‌جای یک بار چک کردن در شروع اپ، تغییرات را با `registerDefaultNetworkCallback` دنبال کنید



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. ART کد شما را چطور اجرا می‌کند



- **Cloud Profile:** فروشگاه Play، profileهای کاربران یک نسخه را جمع می‌کند و همراه نصب‌های بعدی می‌فرستد
- **Baseline Profile:** فهرست مسیرهای داغ را خودتان داخل اپ می‌فرستید و در زمان نصب استفاده می‌شود. برای نصب‌های بیرون از Play، کتابخانهٔ `profileinstaller` آن را برای dexopt بعدیِ پس‌زمینه به ART می‌دهد
- قالب ruleها: `HSPLcom/example/FeedViewModel;->load()V` — حرف H یعنی Hot، حرف S یعنی Startup و حرف P یعنی Post-startup



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. تله‌های profile

تلهٔ مصاحبه

«*با وجود Cloud Profile دیگر Baseline Profile لازم نیست.*» — Cloud Profile برای هر نسخهٔ جدید زمان می‌برد تا جمع شود، و نصب‌های بیرون از Play اصلاً آن را نمی‌گیرند. Baseline Profile از اولین نصب کار می‌کند.

تلهٔ مصاحبه

«*چرا همه چیز را AOT کامپایل نکنیم؟*» — کامپایل کامل، اپ را روی دیسک بزرگ‌تر و نصب و آپدیت را کندتر می‌کند، و بیشتر کد سرد است. کامپایل مبتنی بر profile همان چیزی را هدف می‌گیرد که کاربر واقعاً اجرا می‌کند.

تلهٔ مصاحبه

«*Baseline Profile را در build دیباگ تست کردم.*» — اپ‌های debuggable از کامپایل مبتنی بر profile استفاده نمی‌کنند. یک build ریلیز غیر debuggable را اندازه بگیرید.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

Baseline و Startup Profile، Macrobenchmark به‌طور عمیق،
Android vitals



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. خط تولید profile: پیکربندی Gradle

```
// :app — the consumer
plugins {
    id("com.android.application")
    id("androidx.baselineprofile")
}
dependencies {
    implementation(libs.androidx.profileinstaller)
    baselineProfile(project(":baselineprofile"))
}

// :baselineprofile — the producer (com.android.test)
android { targetProjectPath = ":app" }
baselineProfile {
    managedDevices += "pixel6Api34"
    useConnectedDevices = false
}
```

- دستور `./gradlew :app:generateBaselineProfile` مولد را اجرا می‌کند (اینجا روی یک Gradle Managed Device) و profileها را در source set اپ می‌نویسد — commitشان کنید
- کتابخانه‌های AndroidX مثل Compose profile خودشان را همراه دارند و با profile شما ادغام می‌شوند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. مولد: مسیرهای واقعی را ضبط کنید

```
@RunWith(AndroidJUnit4::class)
class BaselineProfileGenerator {
    @get:Rule val rule = BaselineProfileRule()

    @Test fun generate() = rule.collect(
        packageName = "com.example.app",
        includeInStartupProfile = true,
    ) {
        pressHome()
        startActivityAndWait()
        // the critical journeys, not just startup
        device.findObject(By.res("feed")).fling(Direction.DOWN)
        device.findObject(By.text("Profile")).click()
    }
}
```

- **Baseline Profile** یعنی کامپایل AOT روی دستگاه. **Startup Profile** را R8 در **زمان build** استفاده می‌کند تا کلاس‌های شروع را در فایل DEX اصلی بگذارد (DEX layout)؛ برای همین R8 باید روشن باشد
- در Compose، مقدار `testTagsAsResourceId = true` را در یک `Modifier.semantics` ریشه بگذارید تا `By.res()` تگ‌های تست را پیدا کند

تلهٔ مصاحبه

«*یک بار ساختیمش.*» — profile مسیرهای کد را توصیف می‌کند و کد هر اسپرینت عوض می‌شود. برای هر ریلیز دوباره بسازید و با benchmark تأیید کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. ثابت کنید profile کار می‌کند

```
@Test fun startupNoCompilation() =
    startup(CompilationMode.None())

@Test fun startupBaselineProfile() = startup(
    CompilationMode.Partial(BaselineProfileMode.Require)
)

private fun startup(mode: CompilationMode) =
    rule.measureRepeated(
        packageName = "com.example.app",
        metrics = listOf(StartupTimingMetric()),
        compilationMode = mode,
        startupMode = StartupMode.COLD,
        iterations = 10,
    ) {
        pressHome()
        startActivityAndWait()
    }
```

- حالت `BaselineProfileMode.Require` اگر profile نصب نشده باشد اجرا را fail می‌کند — «موفقیت» بی‌صدا در کار نیست
- در میدان، `ProfileVerifier.getCompilationStatusAsync()` از `profileinstaller` گزارش می‌دهد که آیا این نصب با profile کامپایل شده یا نه



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. Macrobenchmark: آماده‌سازی اپ هدف

```
// app/build.gradle.kts — the baselineprofile plugin
// creates an equivalent "benchmarkRelease" for you
buildTypes {
    create("benchmark") {
        initWith(getByName("release"))
        signingConfig = signingConfigs.getByName("debug")
        matchingFallbacks += listOf("release")
        isDebuggable = false
    }
}
```

```
<application>
    <profileable android:shell="true" tools:targetApi="29" />
</application>
```

- یک ماژول جدای `com.android.test` اپ نصب‌شده را از پروسسی دیگر و با UiAutomator می‌راند — درست مثل یک کاربر
- اپ هدف باید **غیر debuggable** باشد، R8 روشن داشته باشد و **profileable** باشد تا بشود trace گرفت



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. یک benchmark اسکرول

```
@Test fun scrollFeed() = rule.measureRepeated(
    packageName = "com.example.app",
    metrics = listOf(FrameTimingMetric()),
    compilationMode = CompilationMode.Partial(),
    startupMode = StartupMode.WARM,
    iterations = 10,
    setupBlock = { startActivityAndWait() },
) {
    val feed = device.findObject(By.res("feed"))
    // keep flings away from the gesture-navigation edges
    feed.setGestureMargin(device.displayWidth / 5)
    feed.fling(Direction.DOWN)
    device.waitForIdle()
}
```

- بخش `setupBlock` اندازه‌گیری نمی‌شود: اول به صفحهٔ موردنظر بروید. lambdaی آخر همان مسیری است که اندازه‌گیری می‌شود
- خروجی: صدک‌های ۵۰، ۹۰، ۹۵ و ۹۹ برای هر معیار، به‌علاوهٔ یک trace از Perfetto برای هر تکرار که از داخل Android Studio باز می‌شود



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. معیارها و تله‌ها

معیار	چه چیزی را می‌سنجد
`StartupTimingMetric`	`timeToInitialDisplayMs` و `timeToFullDisplayMs`
`FrameTimingMetric`	`frameDurationCpuMs` و `frameOverrunMs` که دومی API 31 به بعد است
`TraceSectionMetric`	بخش‌های `trace("…")` خودتان
`MemoryUsageMetric` و `PowerMetric`	آزمایشی؛ سنجش برق به دستگاهی با ریل‌های برق نیاز دارد

تلهٔ مصاحبه

«*میانگین FPS را گزارش کن.*» — میانگین jank را پنهان می‌کند. **صدک‌های مدت و overrun فریم** را مقایسه کنید (صدک ۹۰ و ۹۹)، و برای شروع اپ هم TTID و TTFD را.

تلهٔ مصاحبه

«*برای تست کارایی یک شبیه‌ساز پایدار کافی است.*» — زمان‌های شبیه‌ساز سخت‌افزار واقعی را نشان نمی‌دهند و Macrobenchmark به‌طور پیش‌فرض روی شبیه‌ساز خطا می‌دهد. از دستگاه فیزیکی استفاده کنید.

سؤال تکمیلی

«*Macrobenchmark یا Microbenchmark؟*» — Macro برای مسیرهای کاربر (شروع اپ، اسکرول، ناوبری) از بیرون اپ. Micro برای یک تابع داغ در یک حلقه، داخل خود پروسس — مثل یک parser یا الگوریتم diff.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. Android vitals: گوگل چطور قضاوتتان می‌کند

vital اصلی	آستانهٔ رفتار بد
نرخ crash قابل‌درک برای کاربر	۱٫۰۹ درصد در کل
نرخ ANR قابل‌درک برای کاربر	۰٫۴۷ درصد در کل
هر کدام، به تفکیک مدل گوشی	۸ درصد

- vitalهای دیگر: شروع کند cold و warm و hot (روز ۱)، رندر کند، frozen frame (بیشتر از ۷۰۰ میلی‌ثانیه)، partial wake lock بیش از حد
- بالاتر از آستانهٔ کلی، Play ممکن است اپ را کمتر نشان دهد؛ بالاتر از آستانهٔ یک مدل گوشی، کاربران آن مدل ممکن است در صفحهٔ اپ هشدار ببینند
- داده از کاربرانی می‌آید که اشتراک اطلاعات استفاده و عیب‌یابی را پذیرفته‌اند؛ از طریق **Play Developer Reporting API** هم در دسترس است



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. vitals را درست بخوانید

تلهٔ مصاحبه

«*نرخ crash یعنی تعداد crash تقسیم بر تعداد session.*» — سهم **کاربران فعال روزانه‌ای** است که دست‌کم یک crash قابل‌درک دیده‌اند. کاربری که ۵۰ بار crash کند یک بار شمرده می‌شود؛ یک crash نادر که بین کاربران زیادی پخش است خیلی به حساب می‌آید.

تلهٔ مصاحبه

«*نرخ کلی ما خوب است، پس مشکلی نداریم.*» — آستانهٔ ۸ درصد برای هر مدل گوشی جداگانه بررسی می‌شود. یک مدل ارزان و پرطرفدار بالای این عدد می‌تواند در صفحهٔ اپ روی همان مدل هشدار بگیرد.

سؤال تکمیلی

«*از یک خوشهٔ ANR چطور به اصلاح می‌رسید؟*» — Play Console، ANRها را همراه stack ترد اصلی گروه‌بندی می‌کند. traceهای `ApplicationExitInfo` را اضافه کنید (روز ۱) و بعد با یک trace سیستمی بازتولیدش کنید.

باتری هم حساب می‌شود

معیار partial wake lock بیش از حد را مثل نرخ crash و ANR ببینید: برای هر ریلیز رویش هشدار بگذارید، نه بعد از شکایت کاربران.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

حجم به‌عنوان معماری، همگام‌سازی کم‌مصرف، ابزارهای آزمایشگاه و میدان



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. حجم اپ به‌عنوان یک دغدغهٔ معماری

حجم از **وابستگی‌ها و assetها** بزرگ می‌شود، به‌ندرت از کد خودتان — پس همین‌ها را مدیریت کنید:

- هر SDK جدید همراه با diff حجم build ریلیز در PR می‌آید (بودجه در CI: روز ۲۸)؛ برای هر کار فقط یک کتابخانه

گزینهٔ تحویل	مناسب برای	هزینه
ماژول‌های install-time	فقط ماژولار کردن	بدون برد حجمی
ماژول‌های on-demand	فیچرهای بزرگ و کم‌استفاده	دانلود در زمان اجرا، وضعیت نصب، تست
تحویل شرطی	بعضی دستگاه‌ها یا کشورها	ماتریس ریلیز پیچیده
asset packها: install-time، fast-follow، on-demand	مدل‌های بزرگ، رسانه، مرحله‌های بازی	مدیریت چرخهٔ عمر asset

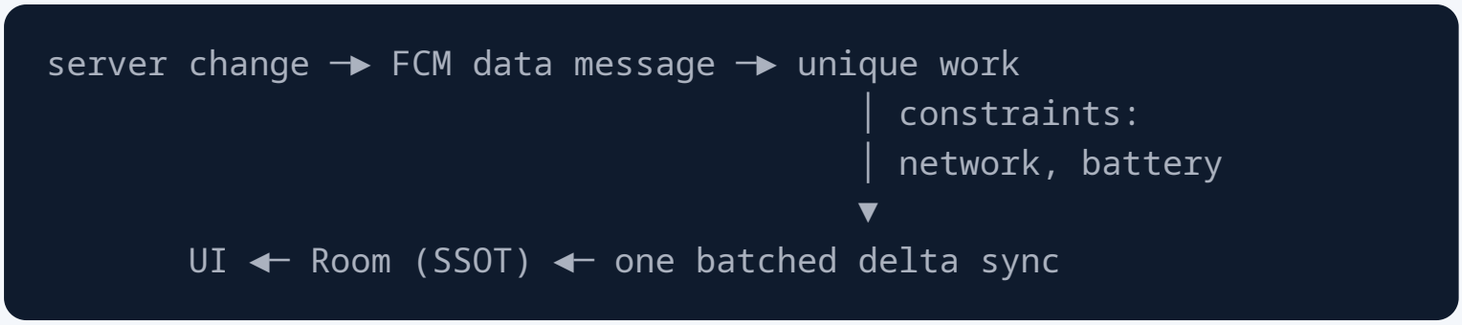
تلهٔ مصاحبه

«برای کوچک کردن اپ، فیچرها را به ماژول‌های on-demand ببر.» — کاربر درست وقتی فیچر را لازم دارد باید منتظر دانلود بماند، و خطاهای نصب و تست به عهدهٔ شماست. فقط برای فیچرهای بزرگ و کم‌استفاده می‌ارزد (روز ۲۵).



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. همگام‌سازی کم‌مصرف برای باتری و شبکه



- **push کنید، poll نکنید:** سرور می‌گوید «چیزی عوض شد»؛ اپ یک بار sync می‌کند (روز ۱۶)
- **یکی کردن:** unique work ده سیگنال را به یک sync تبدیل می‌کند
- **delta** با `since=<cursor>`، به‌جای دانلود کامل؛ backoff نمایی هنگام خطا
- **بخش سنگین را عقب بیندازید** — آپلود رسانه، prefetch — تا شبکهٔ unmetered یا زمان شارژ



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. درخواست sync

```
fun onSyncSignal(workManager: WorkManager) {
    val request = OneTimeWorkRequestBuilder<SyncWorker>()
        .setConstraints(
            Constraints.Builder()
                .setRequiredNetworkType(NetworkType.CONNECTED)
                .setRequiresBatteryNotLow(true)
                .build()
        )
        .setBackoffCriteria(
            BackoffPolicy.EXPONENTIAL, 30, TimeUnit.SECONDS,
        )
        .build()
    workManager.enqueueUniqueWork(
        "sync", ExistingWorkPolicy.KEEP, request,
    )
}
```

تلهٔ مصاحبه

سیاست `KEEP` سیگنالی را که وسط **اجرای** یک sync می‌رسد دور می‌ریزد. در پایان sync دوباره cursor را چک کنید، یا از `APPEND_OR_REPLACE` استفاده کنید.

تلهٔ مصاحبه

«*یک job دوره‌ای ۱۵ دقیقه‌ای پیش‌فرض امن است.*» — این هم poll کردن است: هر اجرا دستگاه و رادیو را بیدار می‌کند. اول sync با push؛ کار دوره‌ای فقط به‌عنوان تور ایمنی کندتر.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. ابزارهای آزمایشگاه و میدان

سؤال	آزمایشگاه، قبل از ریلیز	میدان، بعد از ریلیز
شروع سریع؟	`StartupTimingMetric`	شروع‌های کند در vitals
اسکرول روان؟	`FrameTimingMetric` و Perfetto	JankStats، فریم‌های کند و frozen
profile اعمال شد؟	مقایسهٔ `CompilationMode`ها	`ProfileVerifier`
باتری؟	Power Profiler و `PowerMetric`	wake lockها در vitals
چرا؟	یک trace برای هر تکرار	`ProfilingManager` در اندروید ۱۵ به بعد، `ApplicationExitInfo`

- آزمایشگاه تکرارپذیر است و regression را زود می‌گیرد؛ میدان ترکیب دستگاه‌ها، حجم داده و شرایط حرارتی‌ای را نشان می‌دهد که هیچ‌وقت تست نکرده‌اید



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. بستن حلقه در هر ریلیز

۱. Baseline Profile و Startup Profile را دوباره بسازید؛ Macrobenchmarkها را روی دستگاه فیزیکی اجرا کنید

۲. انتشار مرحله‌ای: vitals را به تفکیک نسخه و مدل گوشی زیر نظر بگیرید (با Reporting API و هشدار)

۳. regression؟ traceهای میدانی و خوشه‌های ANR را بگیرید، در آزمایشگاه بازتولید کنید، اصلاح کنید و یک benchmark اضافه کنید

بودجه‌ها، داشبوردها و فرهنگ کارایی: روز ۲۸.

تلهٔ مصاحبه

«*مانیتورینگ میدانی داریم، پس benchmark لازم نیست.*» — دادهٔ میدانی بعد از آسیب دیدن کاربر می‌رسد و پرنویز است. benchmark آزمایشگاهی جلوی regression را قبل از ریلیز می‌گیرد. به هر دو نیاز دارید.

جواب سینیور در یک جمله

«بایت کمتر بفرست، دستگاه را کمتر بیدار کن، مسیر داغ را از قبل کامپایل کن — و هر ادعا را با benchmark در آزمایشگاه و vitals در میدان ثابت کن.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۱۸ — خلاصهٔ یک‌صفحه‌ای

- هیچ‌کس `.aab` را دانلود نمی‌کند؛ Play برای هر دستگاه split APK می‌سازد. build ریلیز را بسنجید.
- حالت full در R8 از AGP 8.0 پیش‌فرض است؛ keep ruleهای دقیق بنویسید.
- گزینهٔ `isShrinkResources` resourceهای بی‌استفاده را حذف می‌کند.
- انتخاب زبان داخل اپ با bundle یعنی درخواست زبان یا خاموش کردن تقسیم زبان.
- کد native برای Android 15 به بالا باید صفحهٔ ۱۶ کیلوبایتی را پشتیبانی کند.
- باتری: wake lock زمان‌دار (یا هیچ — WorkManager)، شبکهٔ دسته‌ای، push به‌جای poll.
- Baseline Profile یعنی AOT هنگام نصب؛ Startup Profile یعنی DEX layout هنگام build.
- در Macrobenchmark: غیر debuggable، دستگاه فیزیکی، صدک به‌جای میانگین.
- آستانه‌های vitals: نرخ crash برابر ۱٫۰۹ و نرخ ANR برابر ۰٫۴۷ درصد کاربران فعال روزانه؛ ۸ درصد برای هر مدل.
- آزمایشگاه جلوی regression را می‌گیرد؛ میدان اصلاح را ثابت می‌کند.

روز ۱۹ در راه است: امنیت، بخش ۱ — مدل‌سازی تهدید، ذخیره‌سازی امن و مجوزها. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700